

Intro to the Stack and Routing

SWE 4537: Server Programming
Lecture 2.1

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

19 November 2025

Agenda

- 1 The Stack
- 2 Architecture
- 3 Environment Setup
- 4 Routing, REST Design
- 5 Data Locations
- 6 Status Codes
- 7 Class Activity
- 8 Auto Doc
- 9 Summary

Why this Stack?

We are building modern, high-performance web services. We need tools that are fast, standard-compliant, and developer-friendly.

The Language: Python

- 👉 Extremely readable syntax.
- 👉 Massive ecosystem (AI, Data, Web).
- 👉 Synchronous by default (easier to debug), with async capabilities when needed.

The Framework: FastAPI

- 👉 **Fast:** High performance, compared with NodeJS and Go.
- 👉 **Modern:** Based on standard Python type hints.
- 👉 **Automatic Docs:** Generates Swagger UI for free.

The Evolution of Python Web Frameworks

Python has evolved. We aren't using the tools from 2010.

Generation 1: Django (The Monolith)

- ➡ Included everything (HTML templates, DB, Admin).
- ➡ Great for websites, heavy for micro-APIs.
- ➡ *Slow* (Synchronous only).

Generation 2: Flask (The Micro-Framework)

- ➡ Lightweight, simple.
- ➡ You add libraries as needed.
- ➡ Still primarily synchronous.

Generation 3: FastAPI (The Modern Standard)

Built on **Modern Python features** (Type Hints + AsyncI/O). It is designed specifically for building APIs, not websites.

The Secret Sauce: Python Type Hints

FastAPI uses Python's Type Hints to validate data *for you*.

Old Python (Dynamic)

```
def get_user(user_id):  
    # What is user_id?  
    # Is it a string? An int?  
    return db.find(user_id)
```

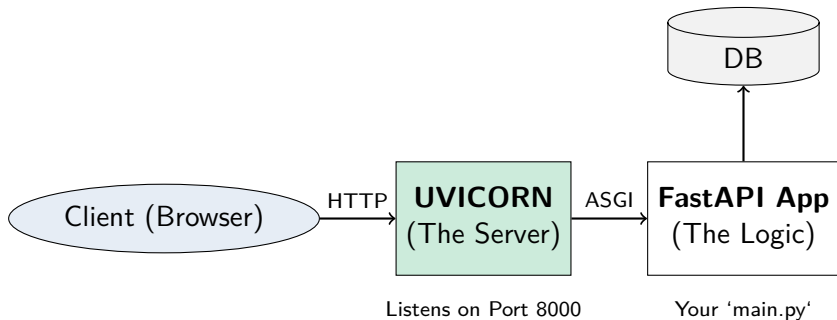
Modern Python (Typed)

```
def get_user(user_id: int) -> dict  
:  
    # Python knows user_id is an  
        INT  
    # Editors give autocomplete  
    return db.find(user_id)
```

Why this matters: FastAPI reads 'user_id: int' and automatically converts "5" to 5, or raises an error for "five".

The Architecture: App vs. Server

Beginners often confuse the **Application** with the **Server**.



- 👉 **Uvicorn** is the "Construction Crew." It handles the raw network bytes.
- 👉 **FastAPI** is the "Blueprint." It decides what to do with the data.
- 👉 **ASGI** is the language they speak to each other.

The "Construction Crew": Uvicorn

The Analogy

If **FastAPI** is the "Blueprint" (the logic of your restaurant),
Uvicorn is the "Staff" (the engine that actually serves the food).

- 👉 **WSGI (Old):** Synchronous (flask, django). One request at a time per worker.
- 👉 **ASGI (New):** Asynchronous Server Gateway Interface. Handles many concurrent connections.

Running the Server

- 👉 Using uvicorn: `uvicorn main:app --reload`
- 👉 Using fastapi: `fastapi dev main.py`

Running the Server

To start the system, we run the Server (Uvicorn) and point it to the App.

```
$ uvicorn main:app --reload
```

main The name of the file ('main.py').

app The name of the variable inside the file ('app = FastAPI()').

--reload Watch for file changes and restart (Dev only!).

or use `fastapi` command to start a dev server

```
$ fastapi dev main.py
```

main The name of the file ('main.py').

dev Development mode

Also watches for file changes and restart.

The “Dependency Hell” Problem

Why do we use `venv` (Virtual Environments)?

Scenario A (Global Install)

- 👉 Project A needs `fastapi` v0.60.
- 👉 Project B needs `fastapi` v0.90.
- 👉 **Conflict!** You can't install both on your laptop globally.

Scenario B (Virtual Env)

- 👉 Project A has a “box” with `fastapi` v0.60.
- 👉 Project B has a “box” with `fastapi` v0.90.
- 👉 **Success!** They run independently.

Development Environment

We never install libraries globally. We use a **Virtual Environment** (Sandbox).

1 Create the sandbox:

- ▶ `python -m venv .venv`

2 Activate the sandbox:

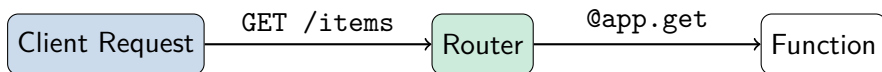
- ▶ Windows: `.\.venv\Scripts\activate`
- ▶ Mac/Linux: `source .venv/bin/activate`

3 Install libraries:

- ▶ `pip install fastapi fastapi[standard] uvicorn`
`python-dotenv`
- ▶ `pip install -r requirements.txt`

What is Routing?

Routing is the mechanism of mapping an **HTTP Request** to a specific **Python Function**.



The Core Question

When the server receives: `GET /users/42`
Which function should run?

Components

- 👉 **Path:** The URL endpoint (e.g., `/users`).
- 👉 **Operation (Method):** The HTTP Verb.
 - ▶ `GET`: Read data.
 - ▶ `POST`: Create data.

The Anatomy of a Route

```
@app.get("/users")
```

- 1 **Decorator (@app):** Tells FastAPI "this function handles web requests."
- 2 **Operation (get):** Corresponds to the HTTP Verb.
 - ▶ GET: Read data (Safe, repeatable).
 - ▶ POST: Create data (Not repeatable).
 - ▶ PUT: Update data (Replace entire object).
 - ▶ DELETE: Delete data.
- 3 **Path ("/users"):** The URL address.

REST Principle 1: Nouns vs. Verbs

In REST (Representational State Transfer), URLs represent **Resources** (Nouns), not Actions (Verbs).

Bad Practice (RPC Style)

- 👉 GET /getAllUsers
- 👉 POST /createUser
- 👉 POST /deleteUser?id=5

Good Practice (REST Style)

- 👉 GET /users
- 👉 POST /users
- 👉 DELETE /users/5

Use HTTP Methods (GET, POST, DELETE) as your verbs!

REST Principle 2: Hierarchy

URLs should reflect the hierarchy of your data.

- 👉 **Collection:** /users (All users)
- 👉 **Item:** /users/42 (Specific user)
- 👉 **Sub-Collection:** /users/42/posts (Posts belonging to user 42)

Design Rule

If a resource cannot exist without a parent, nest it in the URL.

*Example: A Comment cannot exist without a Post →
/posts/1/comments*

Where does data live in a request?

As a backend engineer, you must know where to look for data.

- ➊ **Path Parameter** (/users/**42**):
 - ▶ Used for *Identification*. Mandatory.
- ➋ **Query Parameter** (/users?**sort=desc**):
 - ▶ Used for *Sorting/Filtering*. Optional.
- ➌ **Request Body** (JSON Payload):
 - ▶ Used for *Complex Data* (Creating/Updating). Secure.
- ➍ **Headers** (Authorization: Bearer...):
 - ▶ Used for *Meta-data* (Auth tokens, Content-Type).

Your First API: main.py

Listing 1: A minimal FastAPI app

```
from fastapi import FastAPI

# 1. Create the app instance
app = FastAPI()

# 2. Define a Route
@app.get("/")
def read_root():
    # 3. Return a JSON response
    return {"message": "Hello World"}
```


Path Parameters: “The Specific Resource”

Sometimes the path itself contains variable data (like an ID).

- 👉 Use curly braces {} to define a variable.
- 👉 FastAPI uses Python type hints to convert data automatically.

```
@app.get("/items/{item_id}")
def read_item(item_id: int):
    # item_id is automatically converted to an integer!
    return {"item_id": item_id}
```

Example Request

GET /items/5 → item_id = 5 (int)

GET /items/apple → **Error** (Validation failed)

Validation Error

If you send GET /items/apple, FastAPI automatically returns:
422 Unprocessable Entity: value is not a valid integer.

Path Order Matters

Specific paths must be declared **before** general paths.

```
# Correct Order
@app.get("/users/me")
def read_user_me():
    return {"user_id": "current_user"}

@app.get("/users/{user_id}")
def read_user(user_id: str):
    return {"user_id": user_id}
```

Path Order Matters

Specific paths must be declared **before** general paths.

```
# Correct Order
@app.get("/users/me")
def read_user_me():
    return {"user_id": "current_user"}

@app.get("/users/{user_id}")
def read_user(user_id: str):
    return {"user_id": user_id}
```

Reason:

If you swapped these, /users/me would be caught by user_id and think the ID is "me".

Query Parameters

Parameters that appear after the ? in a URL.

- 👉 Used for sorting, filtering, and pagination.
- 👉 In FastAPI: Function arguments that are **not** in the path.

```
# URL: /items/?skip=0&limit=10
@app.get("/items/")
def read_items(skip: int = 0, limit: int = 10):
    return fake_items_db[skip : skip + limit]
```

Defaults

Since we provided default values (skip = 0, limit = 10), these parameters are **optional**.

Optional Query Parameters

How do we handle parameters that might not be there at all?

We use Python's `None` type or `Optional`.

```
from typing import Optional

@app.get("/items/{item_id}")
def read_item(item_id: str, q: Optional[str] = None):
    if q:
        return {"item_id": item_id, "q": q}
    return {"item_id": item_id}
```

- 👉 `item_id`: Required (Path param)
- 👉 `q`: Optional (Query param, defaults to `None`)

Implementing Request Body (SQLModel)

For complex data (like creating a User), we use **SQL Models**.

```
from sqlmodel import SQLModel

# 1. Define the Schema (The Shape)
class UserCreate(SQLModel):
    username: str
    email: str
    age: int = 18 # Default value

# 2. Use it in the route
@app.post("/users/")
def create_user(user: UserCreate):
    # FastAPI reads the JSON body and converts it to 'user'
    return {"msg": f"Created user {user.username}"}
```

Theory: Idempotency & Safety

Safe Methods (Read-only)

- 👉 GET: Only retrieves data.
- 👉 **Rule:** A GET request should *never* change the database state.

Idempotent Methods

- 👉 PUT, DELETE.
- 👉 **Rule:** Sending the request 1 time or 100 times results in the *same state*.
- 👉 *Deleting user 5 twice is fine (2nd time is just 404).*

The Dangerous One: POST

POST is **NOT Idempotent**. Sending it twice creates **two** resources (e.g., two payments). Be careful!

Status Codes: The API Contract

Don't just return "200 OK" for everything. Be semantic.

👉 2xx: Success

- ▶ **200 OK:** Standard success.
- ▶ **201 Created:** Result of a POST.
- ▶ **204 No Content:** Result of a DELETE.

👉 4xx: Client Error (You messed up)

- ▶ **400 Bad Request:** Generic client error.
- ▶ **401 Unauthorized:** Who are you? (Login needed).
- ▶ **403 Forbidden:** I know you, but you can't do this.
- ▶ **404 Not Found:** Resource doesn't exist.
- ▶ **422 Unprocessable Entity:** Validation error (Wrong type).

Class Activity: Design an API

Scenario: You are building the backend for a Library.

Task: Design the URL routes and decide if you need Path or Query parameters for:

- 1 Get a list of all books.
- 2 Get a specific book by its ISBN.
- 3 Search for books written by "Tolkien".
- 4 Get the first 10 books (pagination).
- 5 Add a new book.
- 6 Remove a book.

Activity Solution

- 1 Get all books:
`GET /books`
- 2 Get specific book:
`GET /books/{isbn}` (Path Param)
- 3 Search by author:
`GET /books?author=Tolkien` (Query Param)
- 4 Pagination:
`GET /books?limit=10&skip=0` (Query Param)
- 5 Add book:
`POST /books` (Data goes in Body - next lecture!)
- 6 Remove book:
`DELETE /books/{isbn}` (Path Param)

The Magic of /docs

Because we used standard Python type hints, FastAPI creates API documentation for us automatically.

👉 Go to: `http://localhost:8000/docs`

👉 Powered by **Swagger UI**.

👉 Interactive: You can run requests directly from the browser.

👉 Alternative:
`http://localhost:8000/redoc`

GET `/items/{item_id}`

Parameters:

`item_id` * integer

`q` string

Summary

- 👉 **The Stack:** Python + FastAPI + Uvicorn.
- 👉 **Setup:** Always use a virtual environment (`.venv`).
- 👉 **Routing:**
 - ▶ Use decorators like `@app.get()`.
 - ▶ **Path Params:** `/items/{id}` (Required variables).
 - ▶ **Query Params:** `/items/?q=...` (Optional filters).
- 👉 **Validation:** Type hints (`int`, `str`) give us free validation and documentation.

Next Lecture: Database Connectivity & ORM (SQLModel)

Questions?

Thank You!

Thank you for your time and attention.
Any questions or clarifications?

